Queries within queries

Hey, it's great to have you back. In this video I'll introduce you to another SQL query: subqueries. A subquery is an SQL query that is nested inside of a larger query. Have you ever seen one of those nesting doll toys? They're also known as matryoshka Russian nesting dolls. Subqueries are a lot like nesting dolls. No, really. Your larger query can have a subquery in it and then that subquery could have a subquery, and then that subquery can have another subquery.

But when you stack them all together, they make one query. With subqueries you can combine different pieces of logic together. Because the logic of your outer query relies on the inner query, you can get more done with a single query. This means all of the logic is in one place, which makes it more efficient and easier to read. The statement containing the subquery can also be called the outer query or the outer select. This makes the subquery the inner query or inner select. The inner query executes first so that the results can be passed on to the outer query to use.

Subqueries can get a little confusing because there are so many layers. But if you keep in mind that the innermost query executes first, it'll be easier to order your subqueries when you want them to execute. Subqueries can also be nested inside all sorts of other queries. Usually you'll find subqueries nested in FROM or WHERE clauses. Let's try out

some common subqueries. We'll start with a subquery in a SELECT statement using the bike-sharing data from an earlier example. For the first statement, let's say we want to compare the number of bikes available at a station to the average number of bikes available.

We're going to use this query to pull the average number of bikes available. Then we're going to incorporate it as a subquery. Now let's build our outer SELECT query. We want to select the station ID and the number of bikes available. Then we'll put the SELECT query that's pulling the average number of bikes inside that outer query by using parentheses. We'll also build FROM into the subquery before closing it with another parenthesis and completing the outer query. The end of the outer join query has AS to show what we want to call this column and a final FROM statement to indicate which table we're referring to.

Now let's run it. And there! We've got a table with both the number of bikes available and the average number of bikes available at different stations. It's really common to see subqueries nested in FROM and WHERE statements. So let's try those next. We could use a FROM statement to calculate the number of rides that have started at each station over time. We'll start with our outer query and input SELECT station_id, name, and number_of_rides.

We'll use AS to tell it how we want the table labeled and FROM to tell it where we're pulling data from. But before we finish that query, we'll add a subquery. We'll put our parenthesis here and then SELECT the start_station_id. Then we can tell it to COUNT the number_of _rides FROM the trip data and group it by the start_station_id. After that, we'll close the subquery with a parenthesis so that we can continue building the outer

query. We'll use AS again and then use INNER JOIN and ON to join it with the station ID data. Finally we'll tell it to put it in descending order.

Let's see what happens when we run that. We now have the number of rides started at each station. One last example. Let's use a WHERE statement. The bike-sharing company has two kinds of users: subscribers and one-time customers. Let's say we wanted a list of stations subscribers used. As always, we start with the outer query. SELECT the station_ id and name FROM the public dataset we're using. This time we'll use a WHERE statement. We'll also use IN so that we can specify multiple values and this WHERE statement. Then we'll put our subquery in the parenthesis. We'll add SELECT, FROM, and WHERE again. But this time we'll tell it that we only want data on specific customers. It's good to note that you can use comparison operators in subqueries, even multiple row operators like IN, ANY, or ALL.

In this case we'll use equals to indicate that we only want the subscriber user data. Now let's run the query and we've got the station id and names for stations that fit our criteria. That's subqueries in action. Subqueries can be challenging. There's a lot of layers to think through and you might find yourself running into errors when you practice. That's totally okay. Having to go through that challenge means you're growing.

If everything was easy, we wouldn't find new ways to grow. For me it's all about how much work and how much time I need to put in to do it. Give yourself time to practice this new concept. Coming up you'll get a chance to use subqueries to aggregate data or you can move on to the weekly challenge. You'll take everything you've learned, using VLOOKUP, different JOINS, and subqueries, and apply it to this upcoming assessment.

We've been doing a lot of complex work. If you want to take a moment to review these videos before moving on, feel free.

Once you've finished a challenge, I'll see you again for our next big learning adventure.

See you soon.